

AhamX Bodhi

Where I Becomes Infinite

Vision Transformers (ViT)

Module 4.1: Vision-Language Models

Prof. Sashikumar Ganesan | IISc Bengaluru | 2026



Session Overview

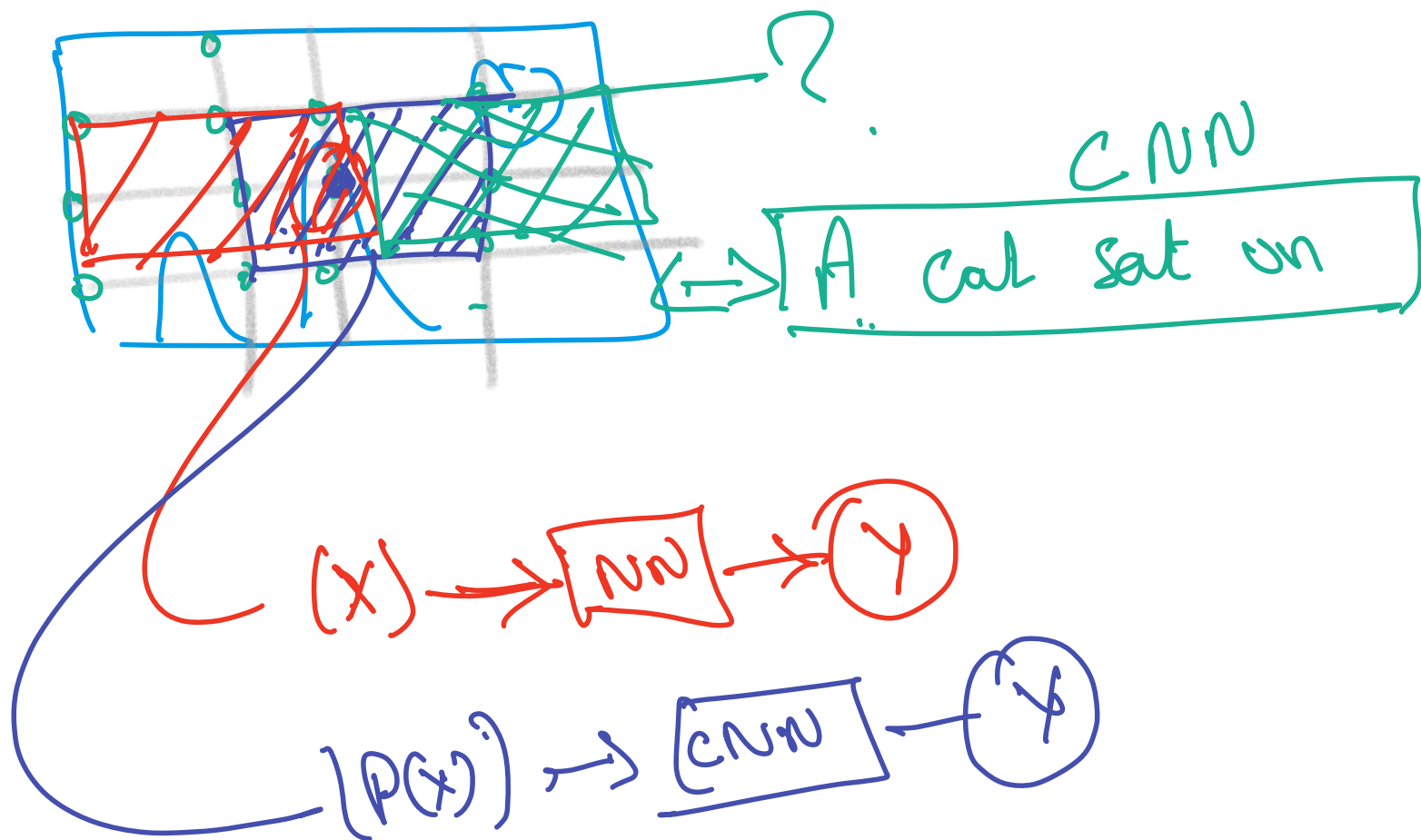
What You Will Learn

- Why CNNs gave way to Transformers for vision
- How ViT converts images into patch sequences
- ViT architecture components in detail
- Variants: DeiT, Swin, DINO, DINOv2
- How AI engineers use ViT in production

Concept at a Glance

- **Duration:** 10 minutes
- **Bloom's Level:** B3 – Apply
- **Difficulty:** L3 – Advanced
- **Module:** 4.1 Vision-Language Models
- **Key paper:** Dosovitskiy et al., 2020







Why CNNs Were Not Enough

CNN Limitations for Multimodal AI

- **Inductive bias:** CNNs assume local spatial structure via convolutions – helpful for ImageNet, limiting for global reasoning
- **Fixed receptive field:** Hard to capture long-range dependencies (e.g., relating a caption word to a distant image region)
- **Poor scalability:** CNN feature maps are hard to align with text token sequences – incompatible with Transformer-based LLMs
- **No sequence representation:** CNNs output spatial feature maps, not sequences – cannot directly interface with attention-based text models

For vision-language alignment, we need a vision encoder that produces a *sequence of vectors* – exactly what a Transformer does naturally.



The Core ViT Idea: Images as Sequences of Patches

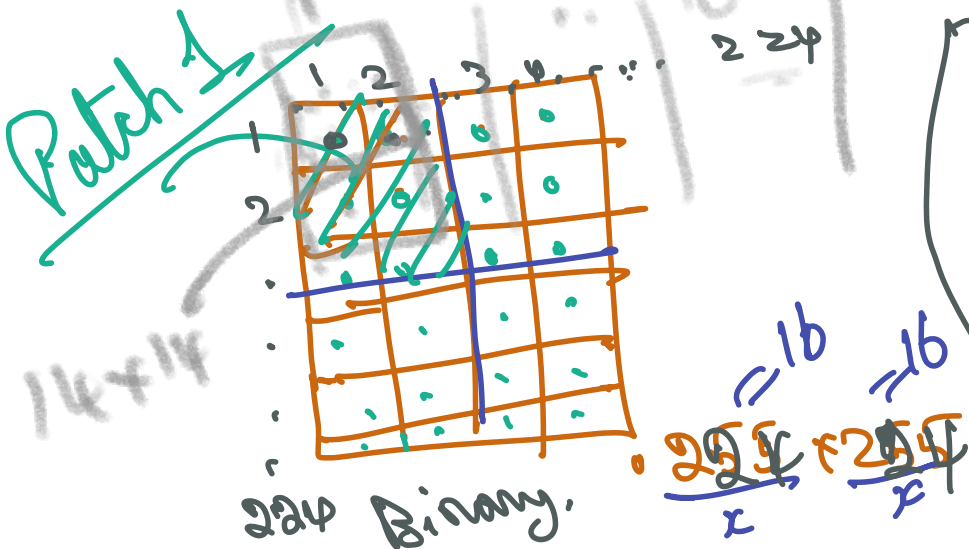
An Image is Worth 16×16 Words (Dosovitskiy et al., 2020)

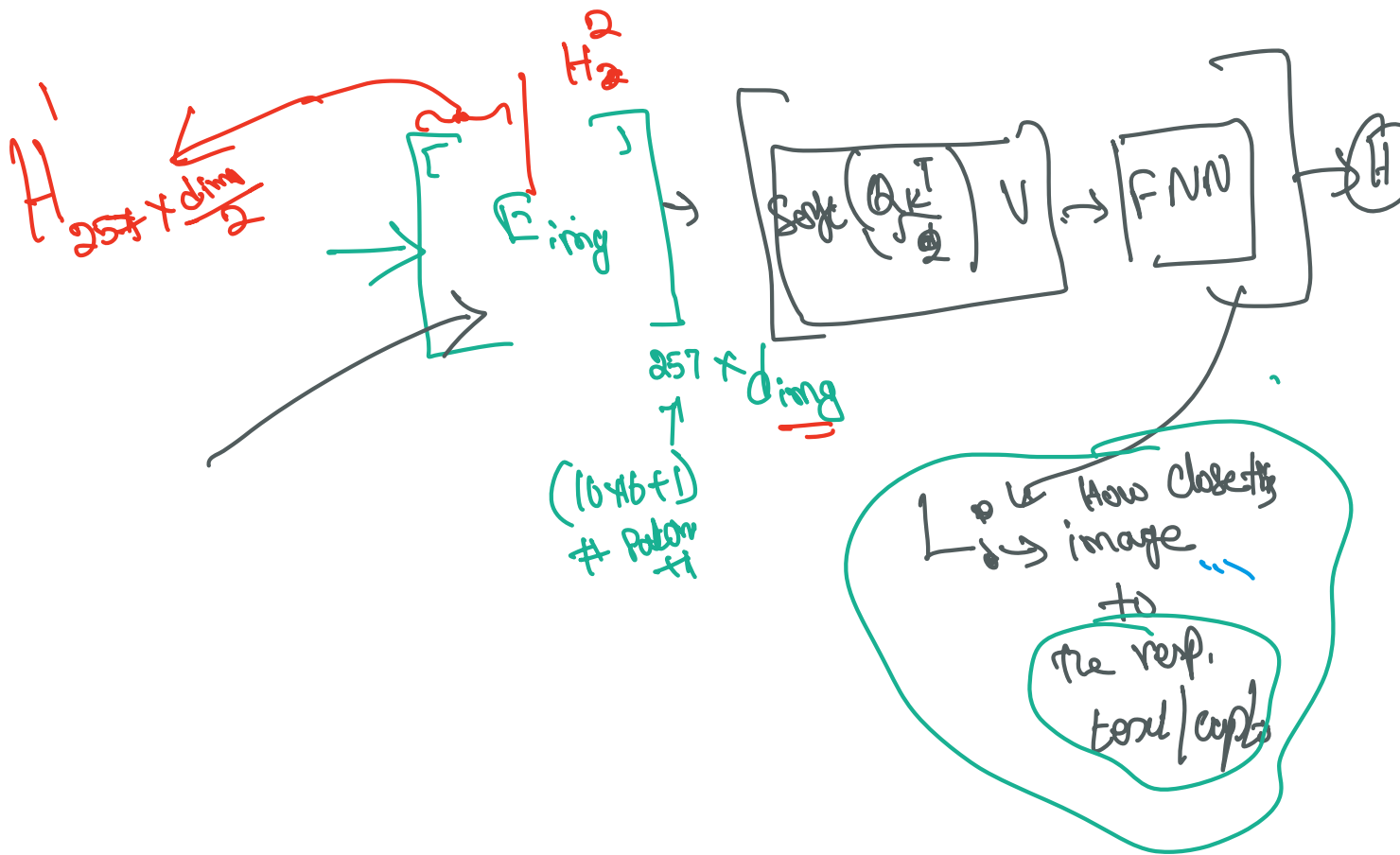
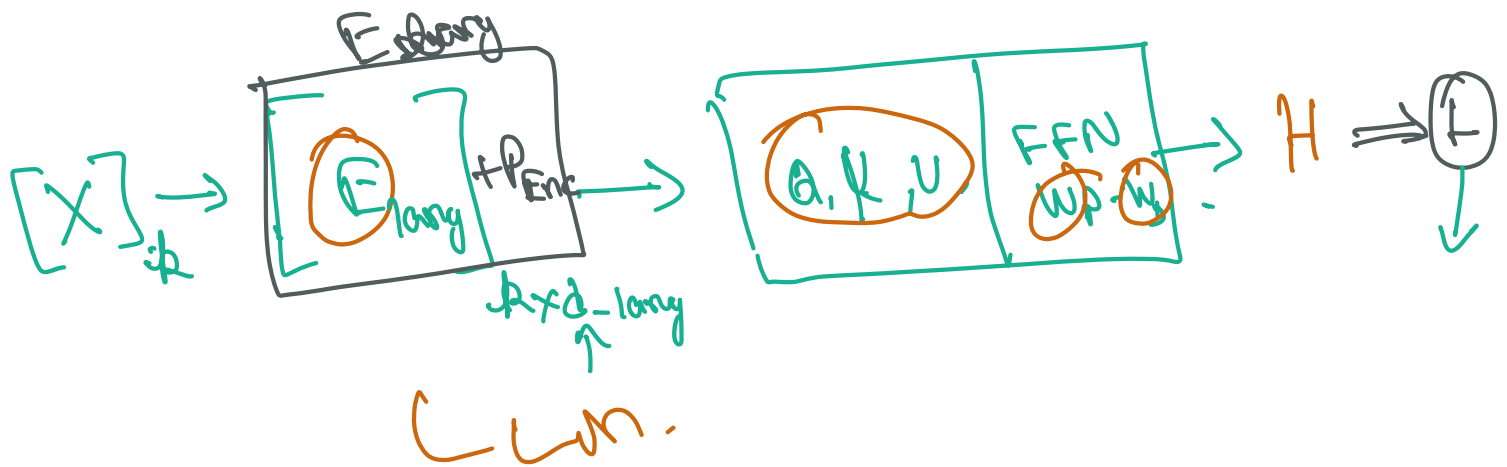
- **Insight:** Treat an image as a sequence of fixed-size patches, just as NLP treats text as a sequence of tokens
- **Patch size:** Typically 16×16 pixels. A 224×224 image becomes $(224/16)^2 = 196$ patches
- **Linear projection:** Each patch (flattened to 768-dim vector) is linearly projected to the embedding dimension
- **CLS token:** A learnable [CLS] token prepended to the patch sequence serves as the global image representation
- **Positional encoding:** 1D learnable positional embeddings added to patch embeddings to encode spatial order

This simple reframing allows any standard Transformer encoder to process images with *zero architectural changes*.

14 x 14

$N = HW/P^2$ patches where P is patch size. For ViT-B/16 on 224×224 :
 $N = 196$, $D = 768$, $L = 12$ layers, 86M parameters.





Example: Grayscale Image

Step 1:

$$X = \begin{bmatrix} P_{11} & P_{12} \\ 1 & 2 & 0 & 1 \\ 0 & 2 & 3 & 4 \\ 1 & 0 & 1 & 2 \\ 3 & 1 & 0 & 1 \\ P_{21} & P_{22} \end{bmatrix} \quad 4 \times 4 = 16 \text{ Pixels}$$

Let us consider: 2×2 Patches = 4 Patches

$$P_1 \Rightarrow [1 \ 2 \ 0 \ 2]$$

$$P_2 \Rightarrow [0 \ 1 \ 3 \ 4]$$

$$P_3 \Rightarrow [1 \ 0 \ 3 \ 1]$$

$$P_4 \Rightarrow [1 \ 2 \ 0 \ 1]_{4 \times 1}$$

Step 2:

Patch Emb.

$$\begin{bmatrix} -P_1- \\ -P_2- \\ -P_3- \\ -P_4- \end{bmatrix}_{4 \times 4}$$

E_{img}^P

$+ b$

to be learned

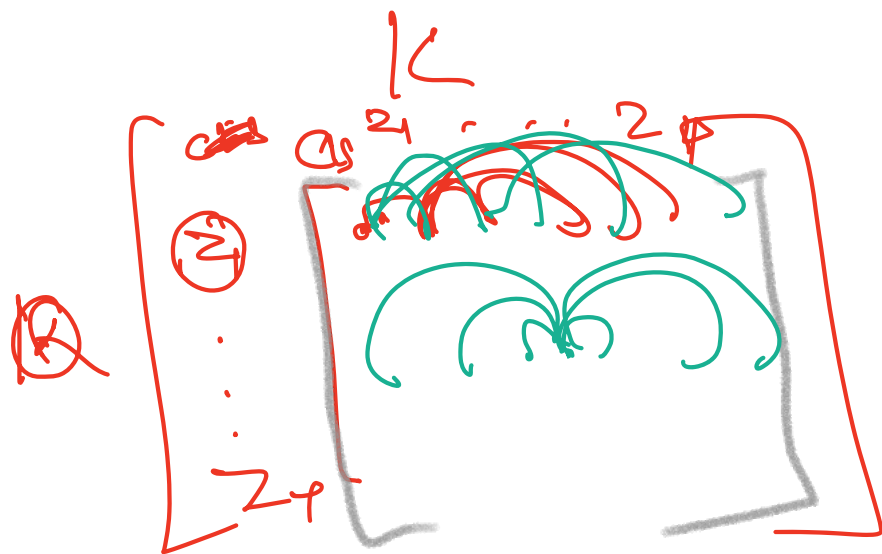
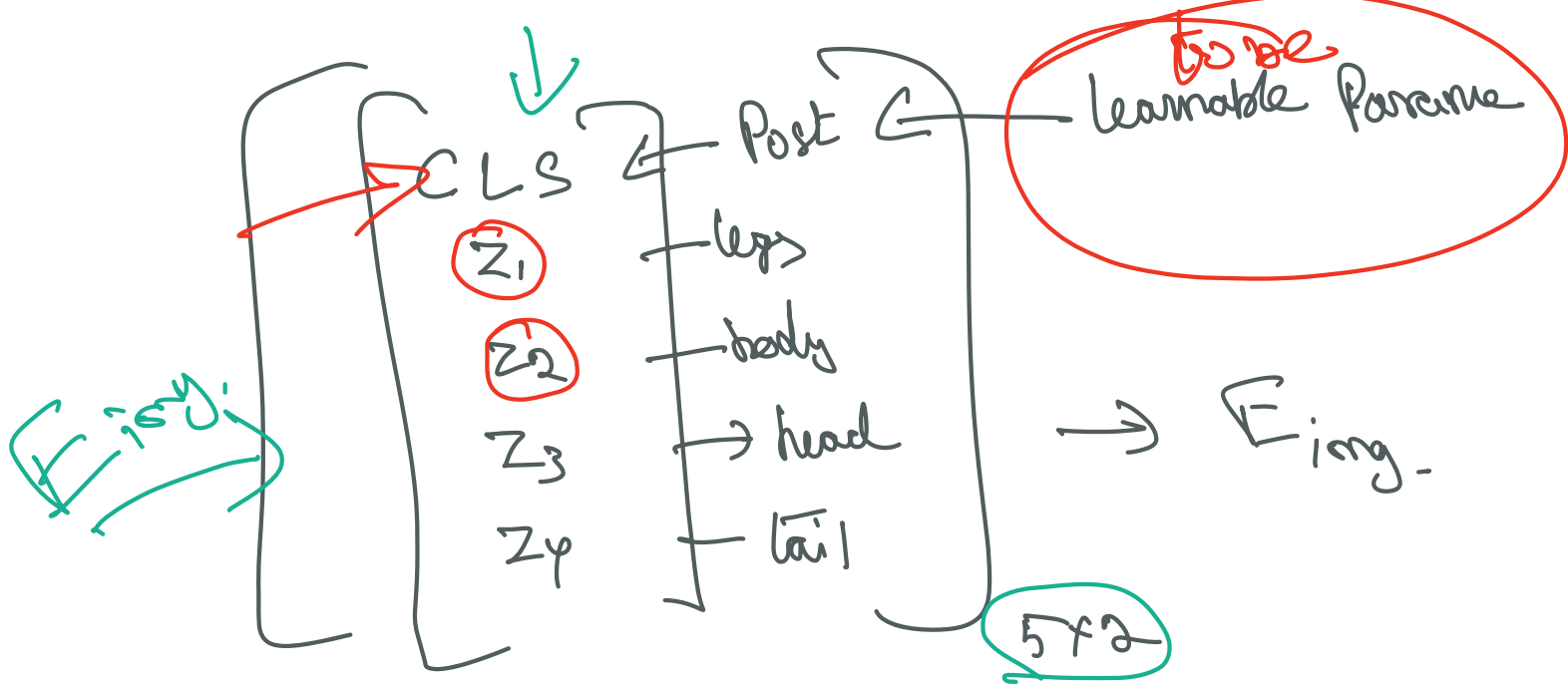
$$4 \times 2 + \epsilon^2 \text{ dim}$$

16, 64

128...

$$\begin{bmatrix} z_1 = [1 \ 3] \\ z_2 = [2 \ 4] \\ z_3 = [1 \ 2] \\ z_4 = [2 \ 3] \end{bmatrix}_{4 \times 2}$$

$4 \times 2 \text{ dim}$



ViT Model Family

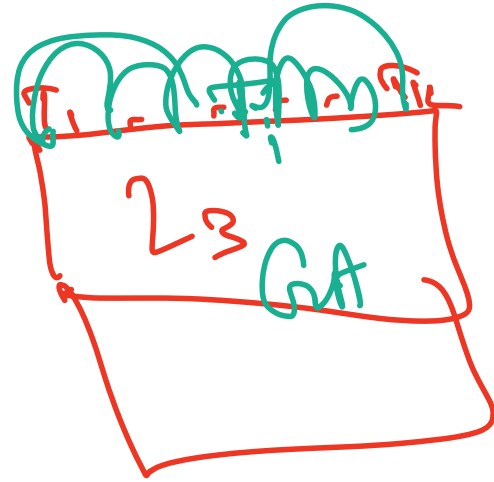
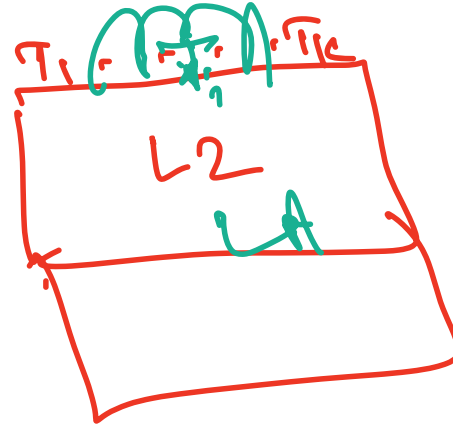
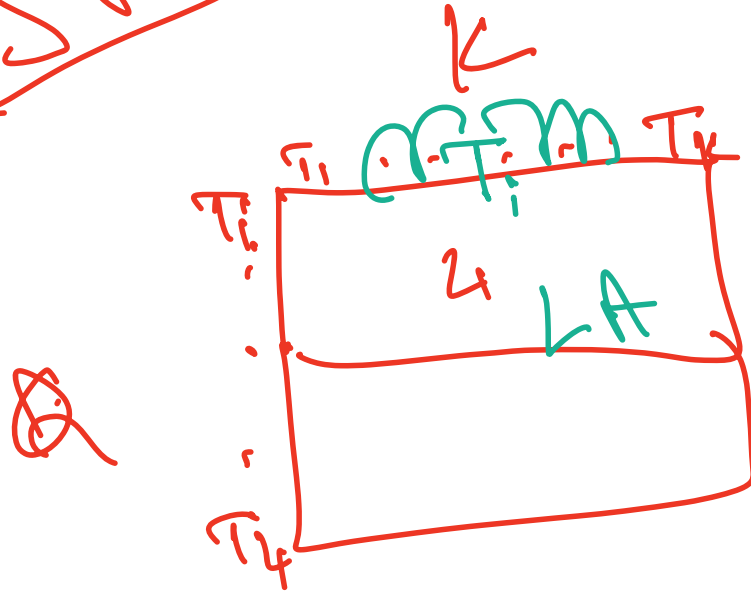


ding

Model	Layers	Embed Dim	Heads	Params	Notes
ViT-Ti/16	12	192	3	5.7M	Tiny, edge deployment
ViT-S/16	12	384	6	22M	Small, efficient
ViT-B/16	12	768	12	86M	Baseline, most common
ViT-L/16	24	1024	16	307M	Large, CLIP default
ViT-H/14	32	1280	16	632M	Huge, highest accuracy

ViT-B/16 and ViT-L/14 are the most widely deployed variants in production VLMs. The "14" suffix means 14×14 pixel patches – finer granularity, more tokens, higher compute

SWT Q! local att $\rightarrow$ global att



Swin Transformer: Hierarchical Vision



Swin: Shifted Window Attention (Liu et al., 2021)

- **Problem with ViT:** Global self-attention over all 196 patches is $O(N^2)$ – expensive for high-resolution images
- **Swin solution:** Local window attention (e.g., 7×7 patches per window) + shifted windows across layers for cross-window communication
- **Hierarchical feature maps:** Like CNNs, Swin produces multi-scale feature pyramids – essential for dense prediction (detection, segmentation)
- **Complexity:** $O(N)$ vs. $O(N^2)$ for standard ViT – significantly faster on high-resolution inputs
- **Adoption:** Swin-T/S/B/L are the backbone of many state-of-the-art detection and segmentation models (Mask2Former, etc.)

SWA in L2m
→ to CNN
more partitions

Swin is preferred when spatial resolution matters – detection, segmentation, and any task requiring localization rather than global classification.



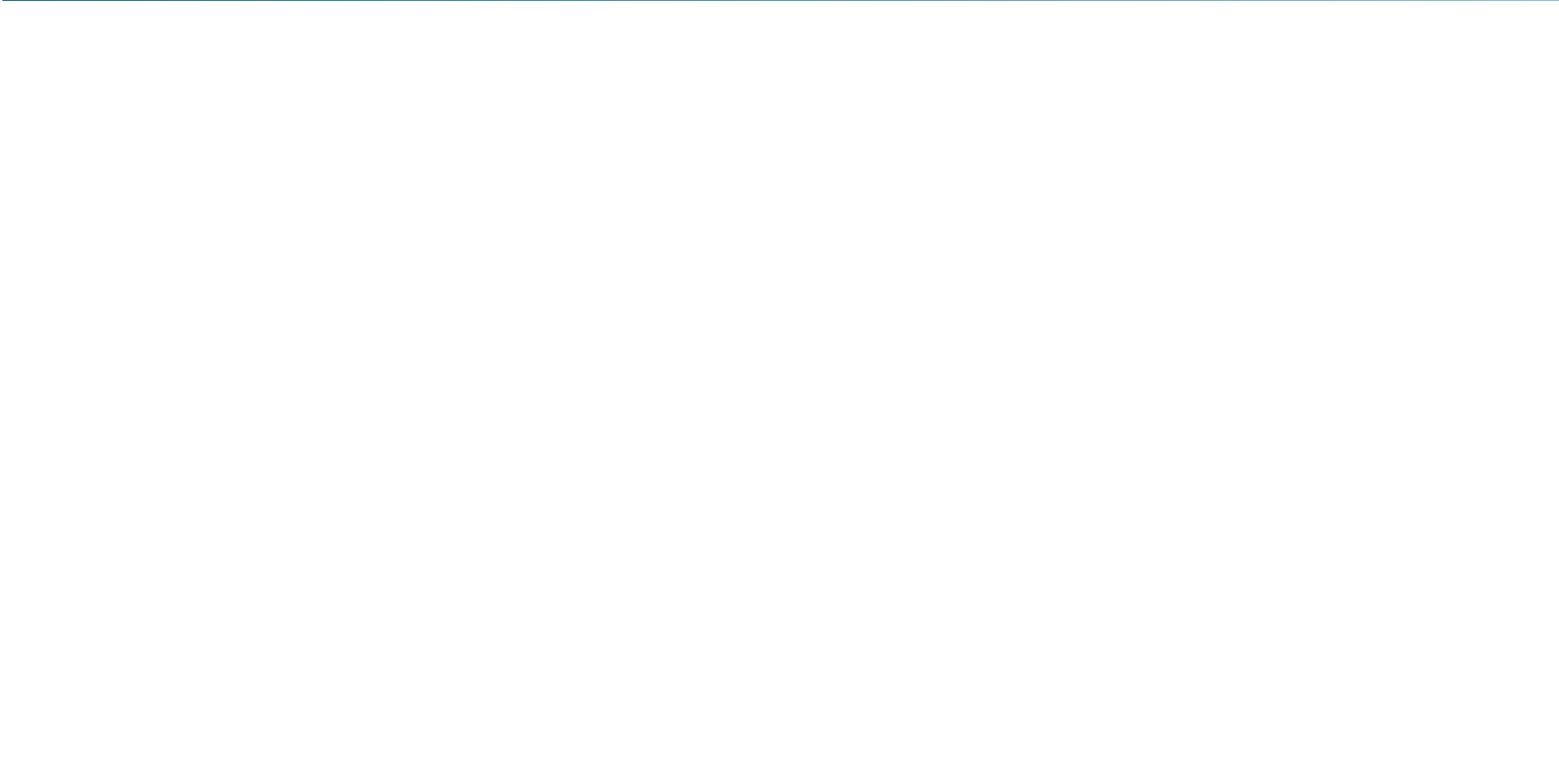
DINO and DINOv2: Self-Supervised ViT

Self-Distillation with No Labels (Caron et al., 2021; Oquab et al., 2023)

- **DINO:** Self-supervised training using knowledge distillation between student and teacher ViT – no labels needed
- **Emergent segmentation:** DINO attention maps naturally segment objects – a remarkable emergent property not trained for
- **DINOv2 (Meta, 2023):** Trained on 142M curated images. Produces universal visual features with no fine-tuning for many tasks
- **Performance:** DINOv2 ViT-G achieves 86.5% on ImageNet linear probe – competitive with supervised models

Engineer's Insight

DINOv2 features are the preferred visual backbone for many open-source VLMs (InternVL, LLaVA-HR) due to strong zero-shot generalization without task-specific fine-tuning.





Engineering ViT: Token Counts and Memory

The Visual Token Problem in Production VLMs

- A 448×448 image with ViT-L/14 produces $(448/14)^2 = 1024$ visual tokens – each consuming KV-cache memory
- **High-res strategy:** Dynamic tiling – split high-res image into tiles, encode each with ViT, concatenate tokens. LLaVA-1.6 uses up to 2880 tokens per image
- **Token compression:** Resampler (Flamingo), Q-Former (BLIP-2), and C-Abstractor (LLaVA-HR) reduce visual tokens to a fixed budget (e.g., 64-256 tokens)
- **Memory math:** At fp16, ViT-L produces 1024 tokens × 1024 dim = 2MB per image – multiplied by batch size

Visual token compression is an active research area – trading off fine-grained visual detail for inference speed and memory efficiency.



Worked Example: ViT Feature Extraction

Extracting Image Features with ViT-L/14 (CLIP)

AI engineers commonly use pre-trained ViT encoders to extract visual features for downstream tasks:

- Load `openai/clip-vit-large-patch14` via HuggingFace `CLIPVisionModel`
- Pass a preprocessed image through the vision model – output is `last_hidden_state`: shape `[batch, 257, 1024]` (196 patches + 1 CLS token)
- Use `pooler_output` (CLS token) as a 1024-dim global image embedding for retrieval or classification
- Use all 196 patch embeddings for tasks requiring spatial awareness (VQA, grounding)

Key API Pattern

- `CLIPVisionModel` for features only; `CLIPModel` for joint text-image scoring
- Always normalize embeddings before cosine similarity: `F.normalize(features, dim=-1)`



GenAI Application: ViT as the Visual Backbone of CLIP

CLIP's Vision Encoder – ViT-L/14 in Production

- CLIP (OpenAI, 2021) uses ViT as its image encoder trained with contrastive loss against text descriptions
- ViT-L/14@336px is the largest CLIP vision encoder – used in DALL-E 3, Stable Diffusion, and GPT-4V
- **Zero-shot classification:** Encode image, encode class name prompts, find nearest text – no task-specific training
- **Image retrieval:** Pinterest, Shopify, and Google Lens use CLIP-based image retrieval at billion-scale with ANN (approximate nearest neighbor) indexing
- **Conditioning diffusion:** CLIP image embeddings condition DALL-E 2's prior model to guide image generation toward specific visual styles

ViT-L/14 has become the default visual backbone for production multimodal AI – it is the “BERT of vision.”



GenAI Application: Visual Search at Scale

Building Visual Search with DINOv2

- **Use case:** Fashion e-commerce – find visually similar products from a photo
- **Pipeline:** DINOv2 ViT-B/14 encodes product catalog images into 768-dim vectors, stored in Qdrant or FAISS; at query time, user uploads a photo, encode with same ViT, retrieve top-k nearest neighbors
- **Scale:** Pinterest Lens searches 200B+ images using ViT-based embeddings
- **Quality advantage:** DINOv2 features outperform CLIP for fine-grained visual similarity because they were not constrained to align with text

Engineering Decision

Use CLIP embeddings when text-image alignment matters (e.g., search by text query). Use DINOv2 when pure visual similarity matters (e.g., “find images that look like this”).

State-of-the-Art ViT Models (2024--2025)



Model	Training	ImageNet	Best Use
DINOv2 ViT-G	Self-supervised	86.5%	Universal visual features
CLIP ViT-L/14	Contrastive (400M)	75.3% zero-shot	Text-image alignment
SigLIP-SO/400M	Sigmoid contrastive	83.1%	VLM visual encoders
EVA-CLIP-18B	Contrastive (2B)	83.4% zero-shot	Largest CLIP variant
SAM ViT-H	Mask prediction	–	Interactive segmentation

SigLIP (Google, 2023) replaces softmax contrastive loss with sigmoid – more efficient for large batches and better performance on downstream VLM tasks.



Key Takeaways

What You Should Remember

- ViT treats images as sequences of flattened patches, making them directly compatible with Transformer encoders and LLM token spaces
- ViT-B/16 (86M) and ViT-L/14 (307M) are the workhorse encoders in production VLMs
- Swin Transformer adds hierarchical feature maps for dense prediction tasks; DINO/DINOv2 enable self-supervised universal visual features
- Visual token count is a key engineering variable: more tokens = better detail but higher memory and latency
- For production: use SigLIP for VLMs, DINOv2 for visual similarity, CLIP for text-image retrieval

AhamX Bodhi

Where I Becomes Infinite